SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1014	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Arunprasath R	**Reg. No.:**	192524077
Date:	23/10/2026	**Duration:**	60 minutes

Code of Conduct

I Arunprasath R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Arunprasath R

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Intelligent Academic Timetable Conflict Resolution Platform

Industry Context

Universities must schedule hundreds of courses, faculty members, classrooms, laboratories, and student batches. Manual timetable preparation can create conflicts and inefficient room utilization.

Problem Statement

Design and develop an intelligent timetable platform that automatically detects scheduling conflicts and generates feasible timetables based on faculty availability, classroom capacity, laboratory requirements, course combinations, and student batches.

Objectives

- Manage academic scheduling constraints.
- Detect faculty conflicts.
- Detect classroom conflicts.
- Handle laboratory requirements.
- Optimize room utilization.
- Support timetable modifications.

Expected Outcomes

- Conflict-free timetables.
- Reduced administrative effort.
- Better classroom utilization.
- Faster timetable generation.5
- Improved academic scheduling.

GIT & DOCKER ASSIGNMENT
Intelligent Academic Timetable Conflict Resolution Platform

1. Proposed Technology Stack

Component	Technology	Purpose
Frontend	HTML, CSS, Bootstrap	Display and manage timetables
Backend	Python Flask	Handle application logic
Database	SQLite	Store courses, faculty, rooms, and schedules
Version Control	Git	Track changes and manage branches
Remote Repository	GitHub	Store and share project source code
Containerization	Docker	Package the application
Container Orchestration	Docker Compose	Configure and run the application
Scheduling Algorithm	Constraint-based scheduling	Generate timetables and detect conflicts

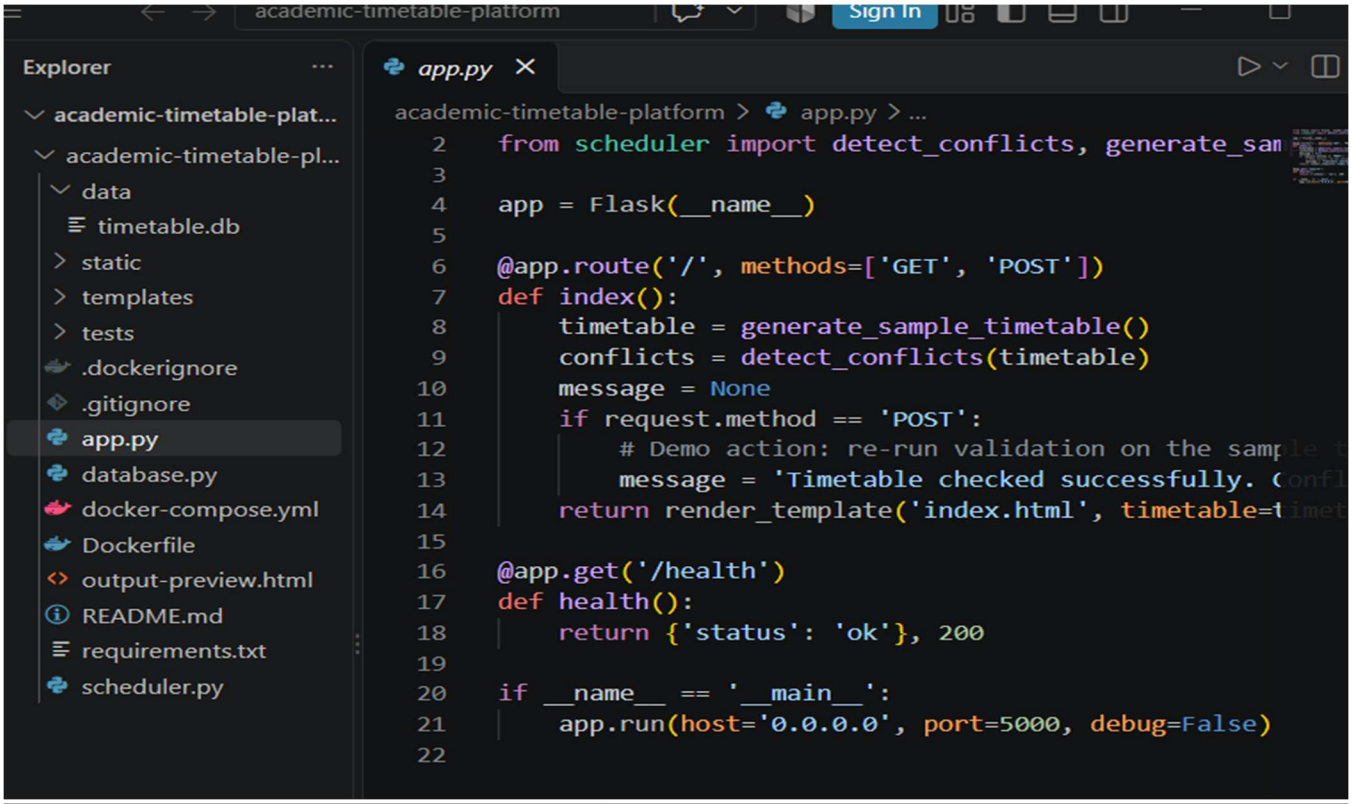
Application:

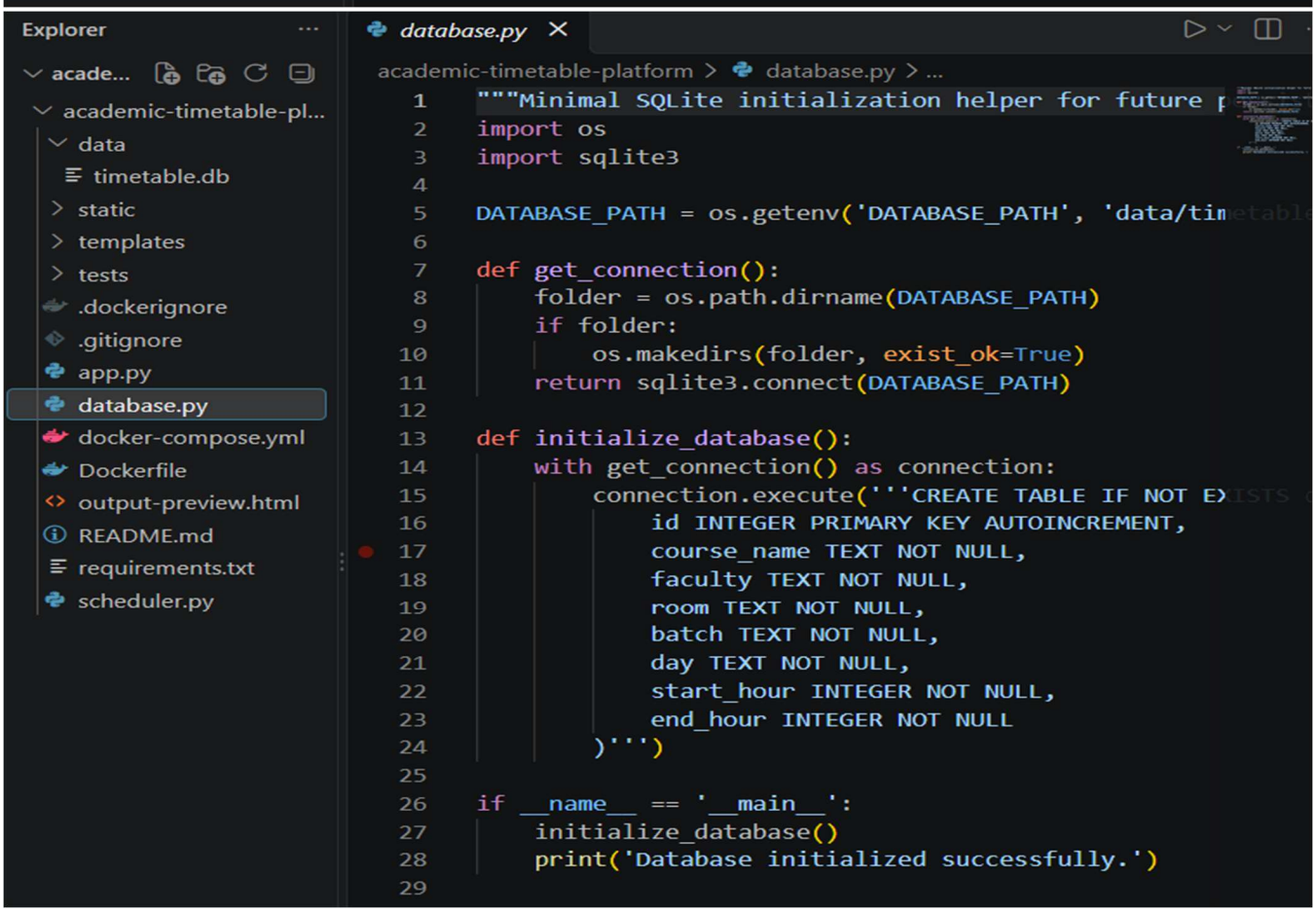
1. The administrator enters course, faculty, classroom, and laboratory details.
2. The application collects the scheduling requirements.
3. The scheduling algorithm assigns courses to available time slots.
4. The application checks for faculty, classroom, and student-batch conflicts.
5. A feasible timetable is displayed.
6. The administrator can modify the schedule when necessary.
7. Docker runs the application in a consistent environment.

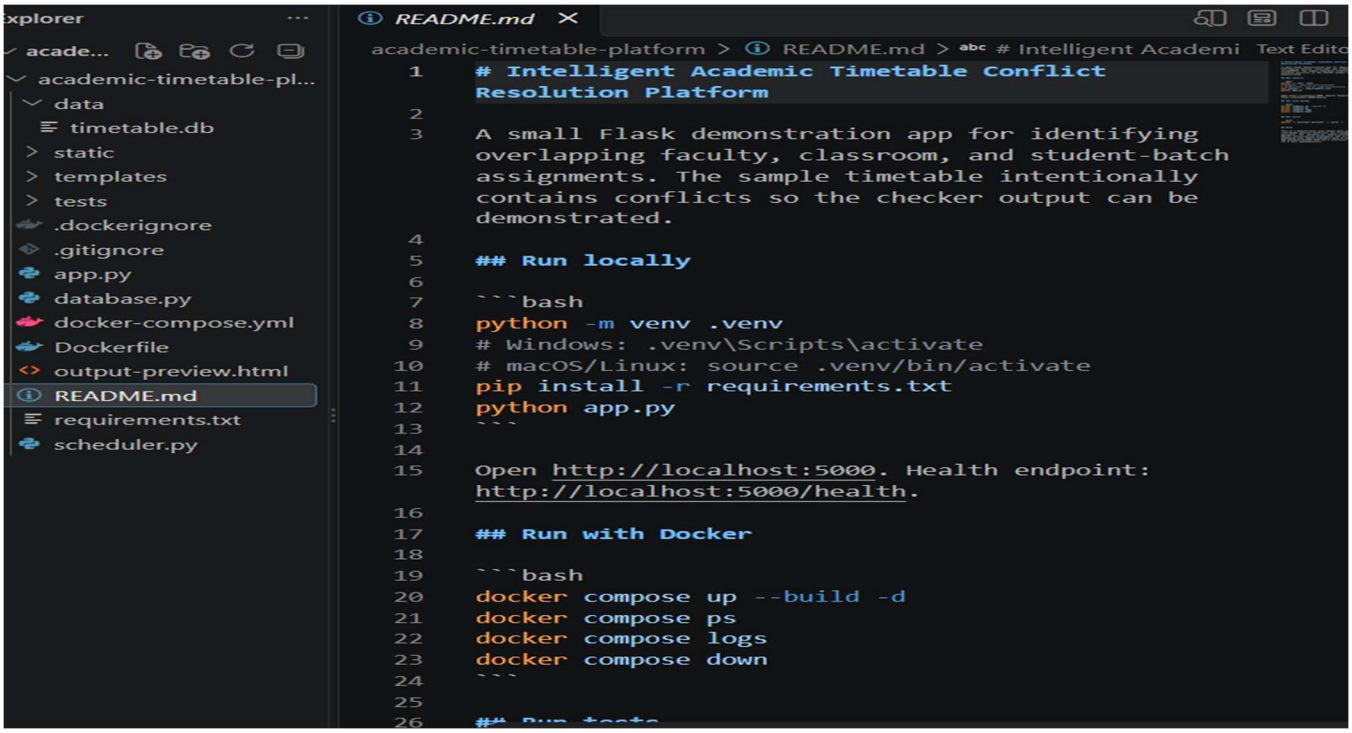
2. Project Structure Design

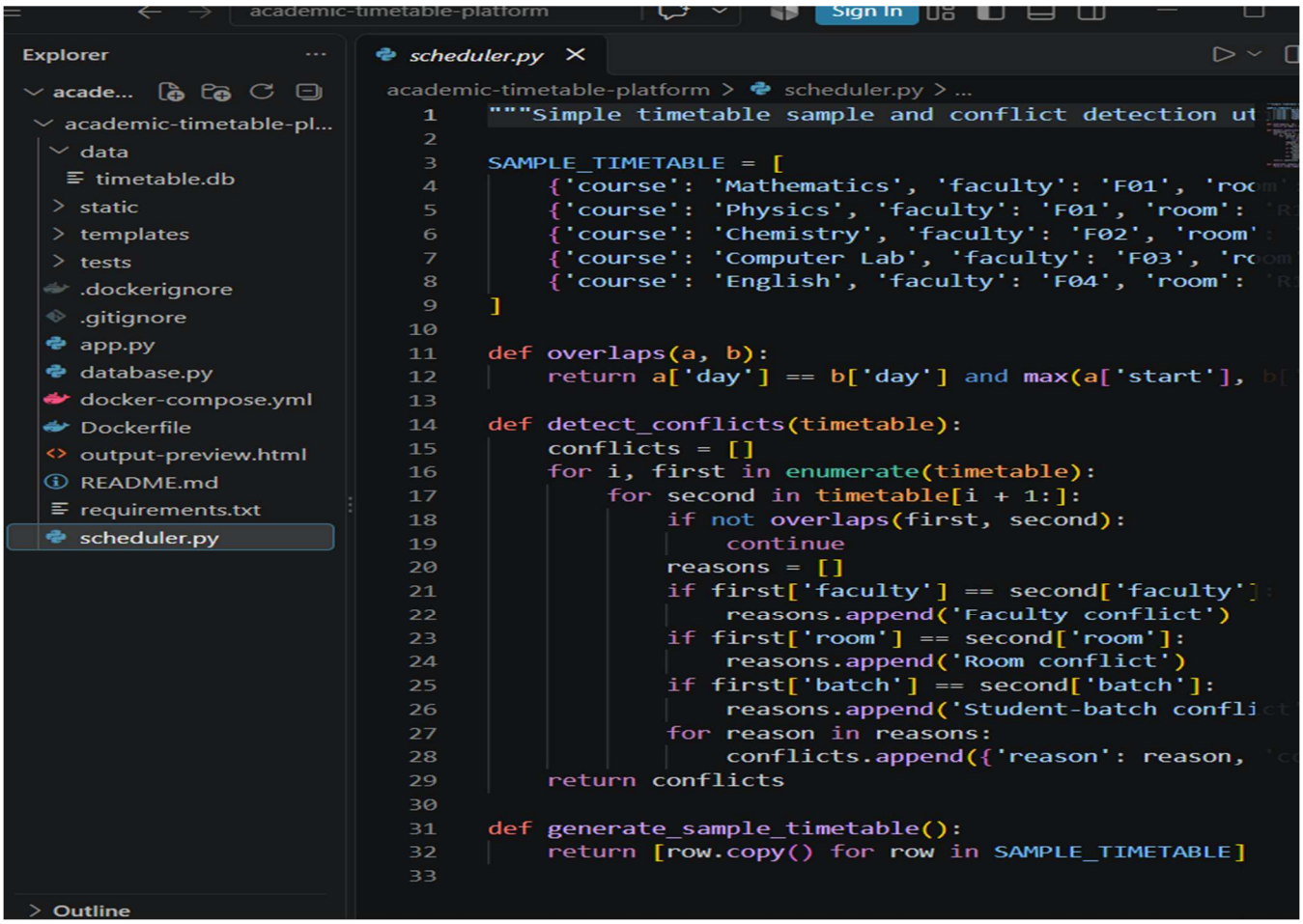
Create a folder named academic-timetable-platform.
Use the following folder structure:

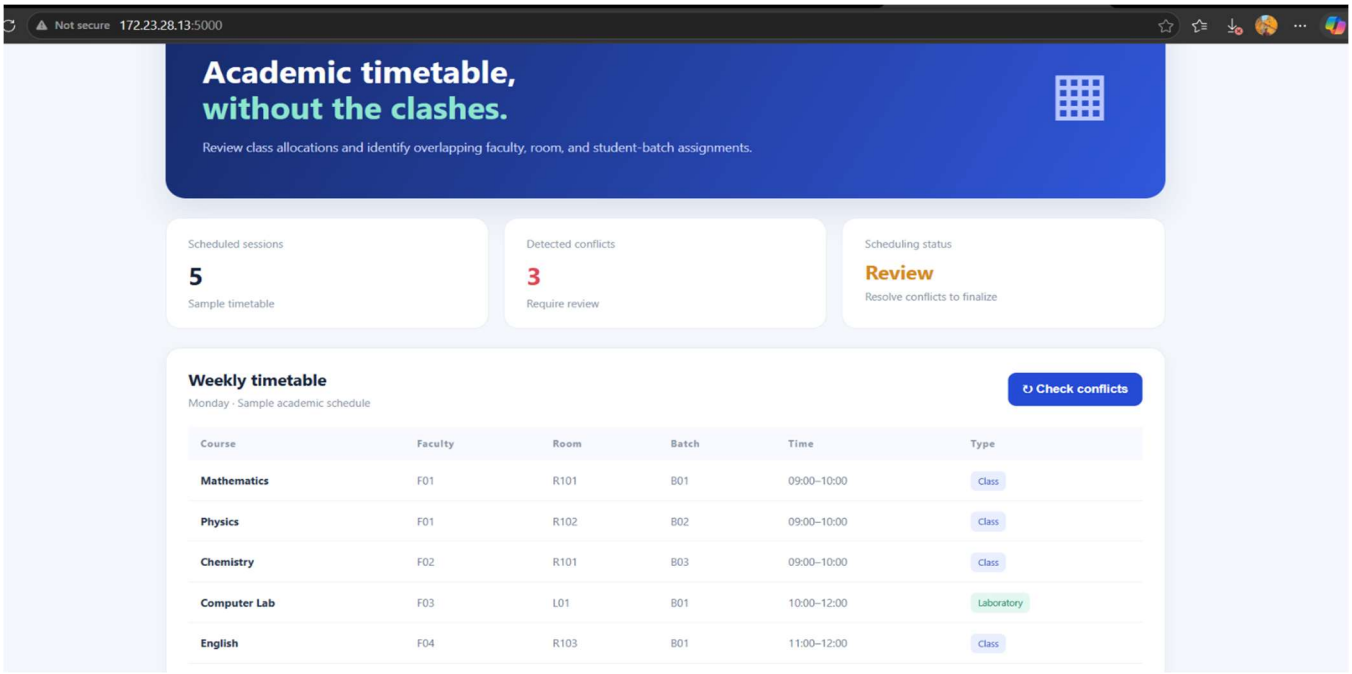
```
academic-timetable-platform/
├── app.py
├── scheduler.py
├── database.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── README.md
├── templates/
│   ├── index.html
│   └── timetable.html
├── static/
│   └── style.css
├── data/
└── tests/
    └── test_scheduler.py
```


academic-timetable-platform
Sign In
Explorer
academic-timetable-plat...
academic-timetable-pl...
data
timetable.db
static
templates
tests
.dockerignore
.gitignore
app.py
database.py
docker-compose.yml
Dockerfile
output-preview.html
README.md
requirements.txt
scheduler.py
app.py
academic-timetable-platform > app.py > ...
from scheduler import detect_conflicts, generate_sam
app = Flask(__name__)
@app.route('/', methods=['GET', 'POST'])
def index():
timetable = generate_sample_timetable()
conflicts = detect_conflicts(timetable)
message = None
if request.method == 'POST':
Demo action: re-run validation on the samp
message = 'Timetable checked successfully.
return render_template('index.html', timetable=t
@app.get('/health')
def health():
return {'status': 'ok'}, 200
if __name__ == '__main__':
app.run(host='0.0.0.0', port=5000, debug=False)


Explorer
acade...
academic-timetable-pl...
data
timetable.db
static
templates
tests
.dockerignore
.gitignore
app.py
database.py
docker-compose.yml
Dockerfile
output-preview.html
README.md
requirements.txt
scheduler.py
database.py
academic-timetable-platform > database.py > ...
"""Minimal SQLite initialization helper for future p
import os
import sqlite3
DATABASE_PATH = os.getenv('DATABASE_PATH', 'data/tim
def get_connection():
folder = os.path.dirname(DATABASE_PATH)
if folder:
os.makedirs(folder, exist_ok=True)
return sqlite3.connect(DATABASE_PATH)
def initialize_database():
with get_connection() as connection:
connection.execute('''CREATE TABLE IF NOT E
id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
faculty TEXT NOT NULL,
room TEXT NOT NULL,
batch TEXT NOT NULL,
day TEXT NOT NULL,
start_hour INTEGER NOT NULL,
end_hour INTEGER NOT NULL
)''')
if __name__ == '__main__':
initialize_database()
print('Database initialized successfully.')


README.md
academic-timetable-platform > README.md > # Intelligent Academi
academic-timetable-platform
data
timetable.db
static
templates
tests
.dockerignore
.gitignore
app.py
database.py
docker-compose.yml
Dockerfile
output-preview.html
README.md
requirements.txt
scheduler.py
Intelligent Academic Timetable Conflict Resolution Platform
A small Flask demonstration app for identifying overlapping faculty, classroom, and student-batch assignments. The sample timetable intentionally contains conflicts so the checker output can be demonstrated.
Run locally
```bash
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
python app.py
```
Open http://localhost:5000. Health endpoint: http://localhost:5000/health.
Run with Docker
```bash
docker compose up --build -d
docker compose ps
docker compose logs
docker compose down
```


Explorer
scheduler.py
academic-timetable-platform > scheduler.py > ...
"""Simple timetable sample and conflict detection ut
SAMPLE_TIMETABLE = [
{'course': 'Mathematics', 'faculty': 'F01', 'roo
{'course': 'Physics', 'faculty': 'F01', 'room':
{'course': 'Chemistry', 'faculty': 'F02', 'room'
{'course': 'Computer Lab', 'faculty': 'F03', 'ro
{'course': 'English', 'faculty': 'F04', 'room':
]
def overlaps(a, b):
return a['day'] == b['day'] and max(a['start'], b[
def detect_conflicts(timetable):
conflicts = []
for i, first in enumerate(timetable):
for second in timetable[i + 1:]:
if not overlaps(first, second):
continue
reasons = []
if first['faculty'] == second['faculty']:
reasons.append('Faculty conflict')
if first['room'] == second['room']:
reasons.append('Room conflict')
if first['batch'] == second['batch']:
reasons.append('Student-batch conflict
for reason in reasons:
conflicts.append({'reason': reason, 'co
return conflicts
def generate_sample_timetable():
return [row.copy() for row in SAMPLE_TIMETABLE]
Outline



Purpose of each file

File or Folder	Purpose
app.py	Runs the Flask application and handles web requests
scheduler.py	Generates timetables and detects scheduling conflicts
database.py	Creates database tables and manages timetable data
requirements.txt	Lists Python dependencies
Dockerfile	Defines how the application Docker image is built
docker-compose.yml	Configures and starts the application container
.dockerignore	Excludes unnecessary files from the Docker build context
.gitignore	Prevents unnecessary files from being tracked by Git
README.md	Contains project documentation and execution instructions
templates/	Stores HTML pages
static/	Stores CSS and other static resources
data/	Stores persistent database files
tests/	Contains automated tests

Important: The .gitignore file is used by Git, whereas .dockerignore is used to exclude files from the Docker build context. They serve different purposes.

3. Git Repository Initialization:

Git allows you to track source-code changes, create branches, and restore earlier versions of your project.

Step 1: Create the project folder

Open PowerShell or the VS Code terminal.

mkdir academic-timetable-platform

cd academic-timetable-platform

Step 2: Initialize Git

```
git init
```

This initializes a local Git repository.

Step 3: Create the initial files

Create the Python files, HTML templates, CSS, and Docker configuration files in VS Code.

Step 4: Create .gitignore

Add the following content:

```
__pycache__/

*.py[cod]

venv/

.env

.pytest_cache/

data/*.db
```

This prevents Python cache files, virtual environments, environment files, test caches, and local database files from being tracked.

If your database contains only harmless sample data that you want to distribute, you can instead include a separate database initialization script in the repository.

Step 5: Make the first commit

```
git add .

git commit -m "Initial project structure"
```

Explanation:

- git add . stages the project files.
- git commit saves the staged changes as a version.
- The commit message describes what was added.

Step 6: Create a GitHub repository

Create an empty repository on GitHub named:

academic-timetable-platform

Connect your local repository to GitHub:

```
git branch -M main

git remote add origin YOUR_GITHUB_REPOSITORY_URL

git push -u origin main
```

Replace YOUR_GITHUB_REPOSITORY_URL with your actual repository URL.

Screenshot evidence: Capture the terminal showing git init, the initial commit, and the successful push to GitHub.

4. Git Branching Strategy:

Branch descriptions

Branch	Purpose
main	Stores stable, tested versions
development	Integrates features before release
feature/scheduler	Develops timetable generation and conflict detection
feature/database	Develops database operations
feature/interface	Develops the web interface

Create the branches

First, create the development branch:

git switch -c development

git push -u origin development

Create the scheduler feature branch:

git switch -c feature/scheduler

After completing and testing the scheduler, merge it into development:

git add .

git commit -m "Implement timetable conflict detection"

git switch development

git merge feature/scheduler

Push the updated development branch:

git push origin development

After testing the integrated application, merge the development branch into main:

git switch main

git merge development

git push origin main

5. Version Control Workflow

Your report should demonstrate meaningful Git operations.

Operation	Command	Purpose
Initialize repository	git init	Creates a local Git repository

Check working tree	git status	Displays modified and untracked files
Stage changes	git add .	Prepares changes for committing
Commit changes	git commit -m "message"	Records a version
Create a branch	git switch -c feature/scheduler	Creates and switches to a feature branch
Change branches	git switch development	Switches to an existing branch
Merge changes	git merge feature/scheduler	Integrates feature changes
View commit history	git log --oneline --graph --all	Displays commits and branch history
Synchronize repository	git push origin development	Uploads local commits to GitHub
Download remote updates	git pull	Retrieves and integrates remote changes

Example: Conflict resolution

Suppose two branches modify the same line in README.md.
Git may report a merge conflict.
The file could contain:

<<<<<<< HEAD

Timetable scheduling using Flask.

=======

Timetable scheduling using Flask and SQLite.

>>>>>>> feature/database

Resolve the conflict by editing the file to retain the intended final content:

Timetable scheduling using Flask and SQLite.

Then run:

git add README.md

git commit -m "Resolve README merge conflict"

Note: A conflict does not occur every time you merge branches. If no conflict occurs naturally, you can demonstrate one using a controlled example and clearly document the steps.

6. Commit History Analysis:

Use meaningful commit messages that describe specific changes.
Example commit history:

a1b2c3d Add initial project structure

b2c3d4e Implement database initialization

c3d4e5f Add timetable generation algorithm

d4e5f6g Implement faculty conflict detection

e5f6g7h Develop timetable web interface

f6g7h8i Add Docker configuration

g7h8i9j Add automated scheduler tests

These are illustrative commit IDs, not actual commits from your repository.
Display your real history with:

git log --oneline --graph --all

Why are good commit messages important?

1. They explain the purpose of each change.
2. They make debugging easier.
3. They help identify when a feature was introduced.
4. They support collaboration between developers.
5. They make project maintenance easier.

Prefer Implement faculty conflict detection over vague messages such as Changes or Update.

7.Docker Environment Design:

Docker packages an application and its dependencies into a container.
For this project, Docker provides a consistent environment for running the Flask application on different computers.
Why use Docker?

- **Portability:** The application can run on systems with Docker installed.
- **Consistency:** Dependencies are installed according to the project configuration.
- **Isolation:** The application runs in its own container environment.
- **Maintainability:** Application dependencies can be managed through configuration files.
- **Deployment:** The application can be deployed without manually installing each Python dependency on the host.
- **Scalability:** The service can be replicated or extended when needed, although a single SQLite-based container is not sufficient for every production scaling scenario.

SQLite runs inside the application container as a database library and file, rather than as a separate database server.
A persistent volume or bind mount should protect the database file from being lost when the container is recreated.

Explanation of each instruction

Instruction	Purpose
FROM python:3.12-slim	Uses a lightweight Python base image
WORKDIR /app	Sets the working directory inside the container
COPY requirements.txt .	Copies the dependency list
RUN pip install ...	Installs the required Python packages
COPY . .	Copies the project files into the image
RUN mkdir -p /app/data	Creates a directory for persistent data

ENV PYTHONUNBUFFERED=1	Sends Python output to logs without unnecessary buffering
EXPOSE 5000	Documents the port used by the application
CMD ["python", "app.py"]	Starts the Flask application

Create requirements.txt

For the initial application, use:

```
Flask
```

SQLite is included with standard Python distributions, so a separate SQLite server package is not needed for this design.

For reproducible submissions, use tested dependency versions or a generated lock file rather than relying indefinitely on unpinned package versions.

Important Flask configuration

Your app.py must start Flask so that it listens on the container's network interface.

For example:

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Without host="0.0.0.0", the application may not be accessible through the published Docker port.

8. Docker Compose Design

Docker Compose defines how the application is configured and started.

Create docker-compose.yml:

```
services:
  timetable-app:
    build: .
    container_name: academic-timetable
    ports:
      - "5000:5000"
    environment:
      FLASK_ENV: production
      DATABASE_PATH: /app/data/timetable.db
    volumes:
```

```
      - timetable-data:/app/data

    restart: unless-stopped

volumes:

  timetable-data:
```

Explanation

Configuration	Purpose
services	Defines the application service
timetable-app	Names the service
build: .	Builds the image using the local Dockerfile
container_name	Gives the container a readable name
ports	Maps host port 5000 to container port 5000
environment	Supplies configuration values to the application
DATABASE_PATH	Specifies the intended database-file location
volumes	Mounts persistent storage at /app/data
restart	Restarts the container automatically under the specified conditions
timetable-data	Defines a named Docker volume

Important implementation detail: Your database.py must read the DATABASE_PATH environment variable for this configuration to work as intended.
For example:

```
import os

import sqlite3

DATABASE_PATH = os.getenv(

    "DATABASE_PATH",

    "data/timetable.db"

)

def get_connection():

    os.makedirs(os.path.dirname(DATABASE_PATH), exist_ok=True)

    return sqlite3.connect(DATABASE_PATH)
```

9. Container Deployment and Testing
After implementing the application, run it through Docker Compose.
Step 1: Check Docker

```
docker --version
```

```
docker compose version
```

These commands display the installed Docker and Compose versions.

Step 2: Build and start the application

From the project root, execute:

```
docker compose up --build -d
```

This command builds the image and starts the container in the background.

Step 3: Check the running container

```
docker compose ps
```

The application should show a running status.

Step 4: View the logs

```
docker compose logs
```

To follow the logs continuously:

```
docker compose logs -f
```

Press Ctrl+C to stop following the logs; this does not normally stop the container.

Step 5: Open the application

Open a browser and visit:

http://localhost:5000

The application should display the timetable interface.

Step 6: Test conflict detection

Use sample scheduling data such as the following:

Course	Faculty	Room	Batch	Time Slot
Mathematics	F01	R101	B01	Monday 9–10 AM
Physics	F01	R102	B02	Monday 9–10 AM
Chemistry	F02	R101	B03	Monday 9–10 AM
Computer Lab	F03	L01	B01	Monday 10–12 AM

Expected checks:

- Mathematics and Physics create a **faculty conflict** because faculty member F01 is assigned to two classes simultaneously.
- Mathematics and Chemistry create a **room conflict** because room R101 is assigned to two classes simultaneously.
- Mathematics and Computer Lab do not overlap in time, so they do not create a time conflict.
- The application should reject or flag conflicting assignments rather than silently accepting them.

The example assumes that the courses and resources are otherwise valid. A complete scheduler should additionally check classroom capacity, laboratory requirements, faculty availability, and student-batch clashes.

10. Challenges and Improvements:

Possible implementation challenges

Challenge	Proposed Solution
Faculty scheduling conflicts	Validate faculty assignments before accepting a timetable
Classroom double-booking	Check room availability for every time slot
Laboratory requirements	Restrict laboratory courses to suitable rooms
Complex scheduling constraints	Apply constraint-based scheduling and backtracking
Git merge conflicts	Communicate changes and resolve conflicting edits
Docker build failures	Verify dependency files and Dockerfile instructions
Database data loss	Store database files in a persistent volume
Different execution environments	Use Docker to standardize the application environment

These are potential challenges to discuss. In your final report, distinguish the problems you actually encountered from those you anticipate.

Future enhancements

1. Introduce an advanced constraint-satisfaction algorithm for generating feasible timetables.
2. Add a timetable optimization mechanism to reduce unused classroom capacity and scheduling gaps.
3. Add administrator and faculty authentication.
4. Provide timetable export in PDF and Excel formats.
5. Replace SQLite with PostgreSQL if concurrent access and larger-scale deployment require a database server.
6. Add automated testing using pytest.
7. Introduce a GitHub Actions workflow to run tests automatically when code is pushed.
8. Deploy the application to a cloud platform that supports Docker containers.

CONCLUSION

The Intelligent Academic Timetable Conflict Resolution Platform provides a systematic approach to managing academic timetables and identifying scheduling conflicts Git and GitHub support version control, collaborative development, and tracking changes throughout the project. Docker simplifies application deployment by packaging the application and its dependencies into a consistent environment. Docker Compose helps manage the application configuration and related services, making deployment easier and more organized.Overall, this project demonstrates how Git and Docker can be integrated into software development to improve code management, application portability, and deployment efficiency. Future enhancements can include automatic timetable optimization, faculty availability management, classroom allocation, and a user-friendly administrative dashboard.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25